

CS218 - Data Structures (DS)

Week – 03 & 04

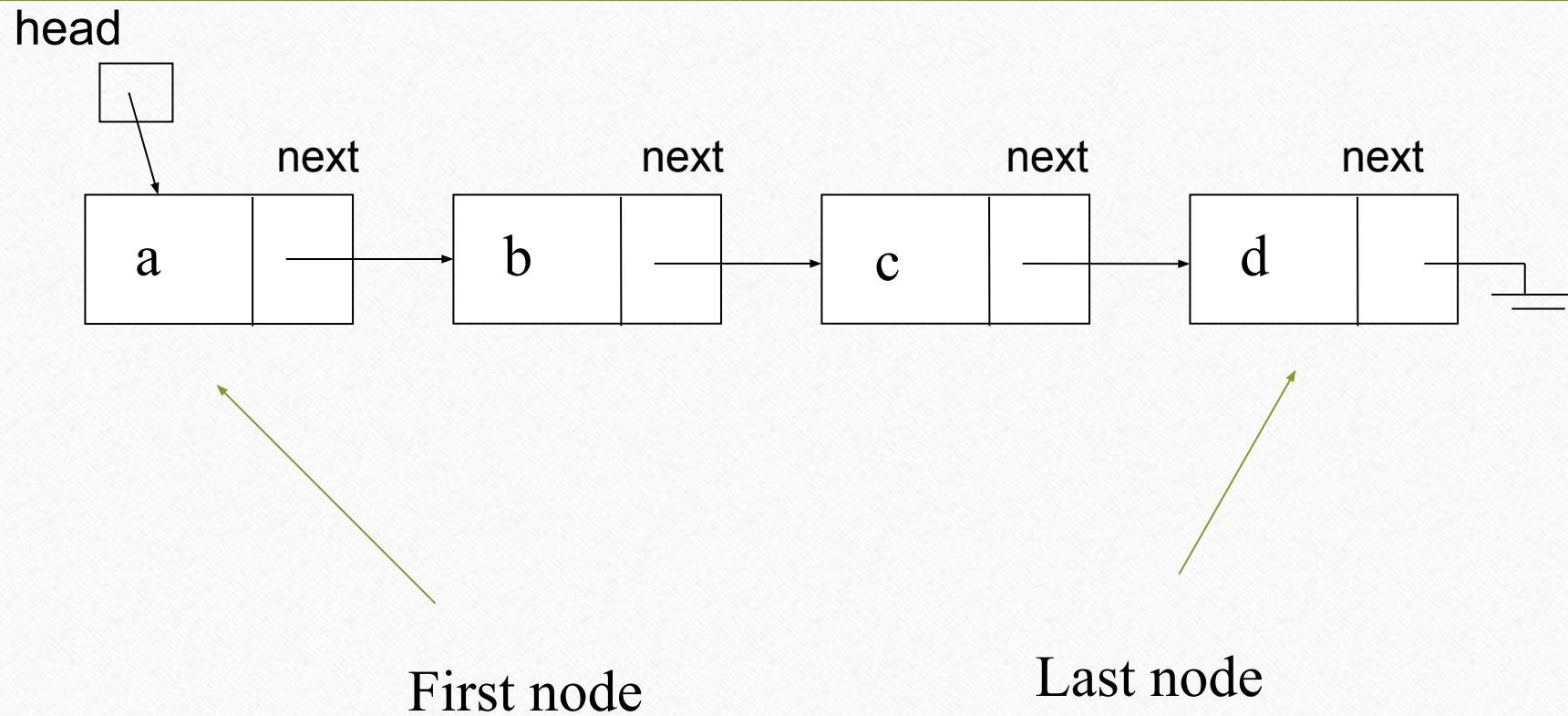
01-05/09-, 2025

Instructor: **Fizza Aqeel**

Linked List Basics

- Linked lists and arrays are similar since they both store collections of data.
- The *array's* features all follow from its strategy of allocating the memory for all its elements in one block of memory.
- *Linked lists* use an entirely different strategy: linked lists allocate memory for each element separately and only when necessary.

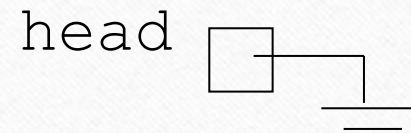
Linked Lists



Empty List

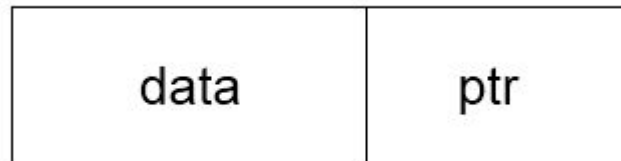
- Empty Linked list is a single pointer having the value NULL.

```
head = NULL;
```



Linked List Basics

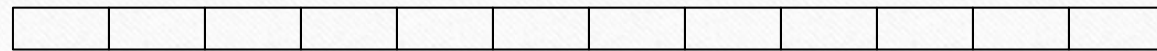
- Each node has (at least) two fields:
 - Data
 - Pointer to the next node



Linked List vs. Array

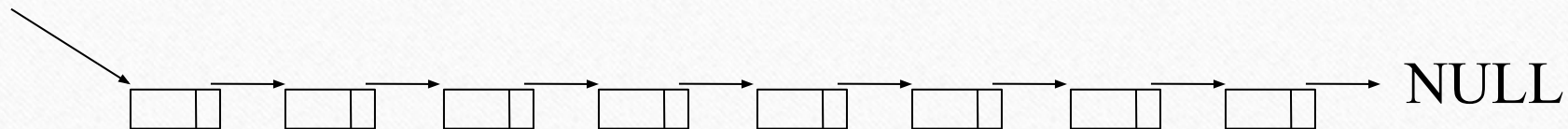
- In a linked list, nodes are not necessarily contiguous in memory (each node is allocated with a separate “new” call)
- Compare this to arrays which are contiguous

Array



Linked List

head



Linked List vs. Array

- **Advantages of Arrays**

- Can directly select any element
- No memory wasted for storing pointers

- **Disadvantages of Arrays:**

- Fixed size (cannot grow or shrink dynamically)
- Need to shift elements to insert an element to the middle
- Memory wasted due to unused elements

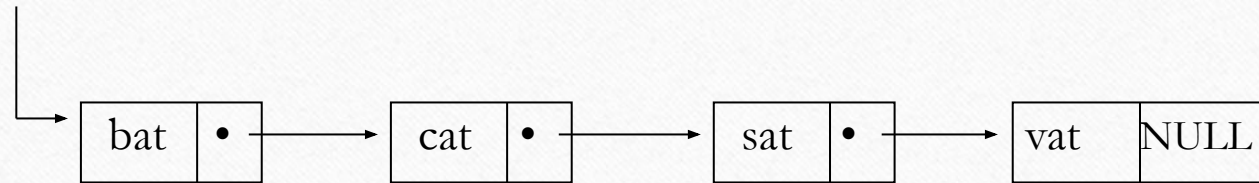
- **Advantages of Linked Lists:**

- Dynamic size (can grow and shrink as needed)
- No need to shift elements to insert into the middle
- Size can exactly match the number of elements (no wasted memory)

- **Disadvantages of Linked Lists**

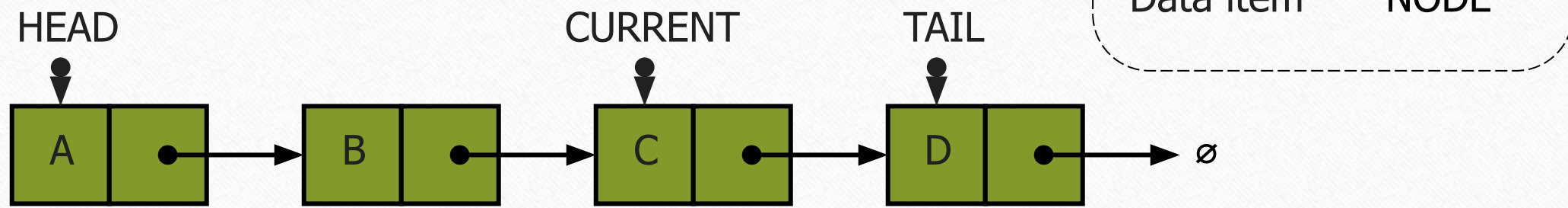
- Cannot directly select any element (need to follow ptrs)
- Extra memory usage for storing pointers

Linked Lists

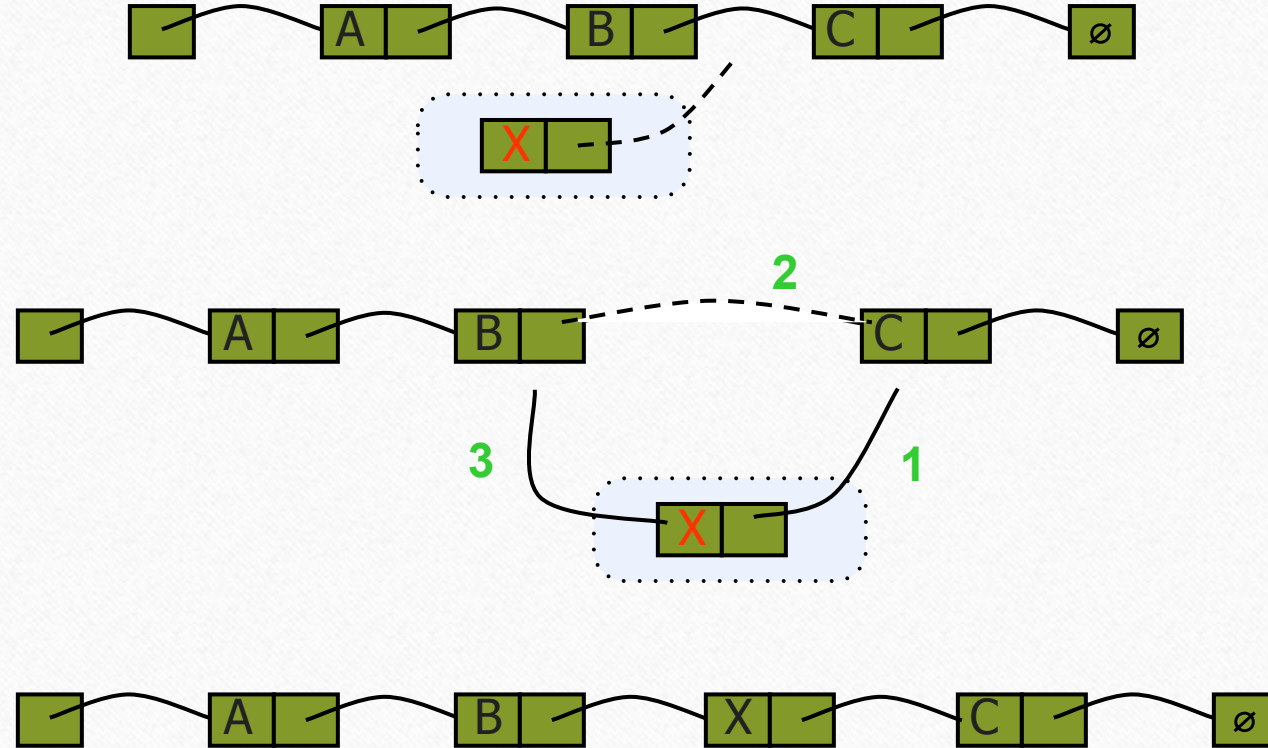


Singly Linked List

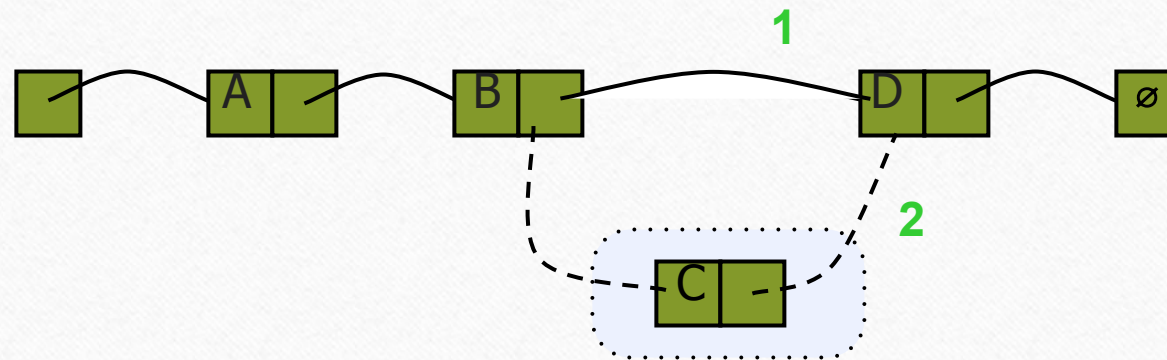
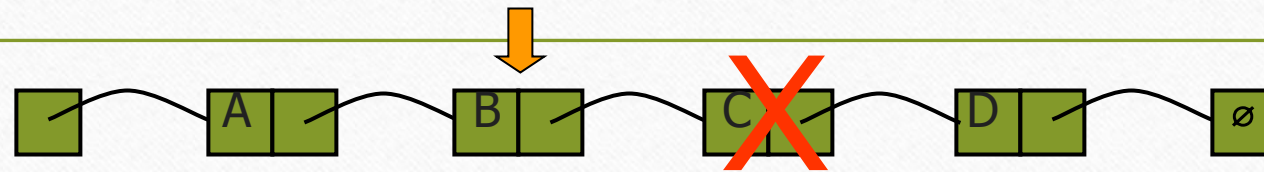
- A singly linked list is a concrete data structure consisting of a series of nodes
- Each node stores
 - Data item
 - Link to the next node



Insertion



Deletion



Singly Linked List (cont..)

```
class Node
```

```
{
```

```
public:
```

```
    int data;
```

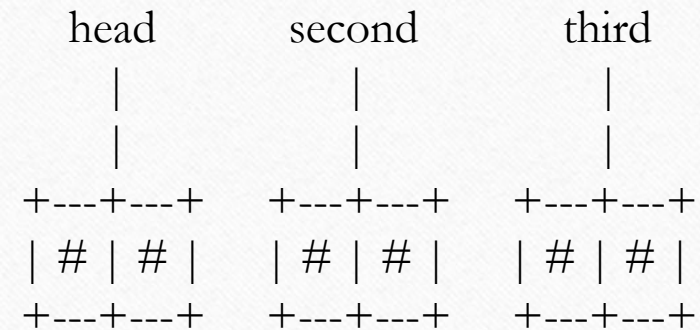
```
    Node* next;    //pointer to another Node
```

```
};
```

```
Node *head = new Node();
```

```
Node *second = new Node();
```

```
Node *third = new Node();
```

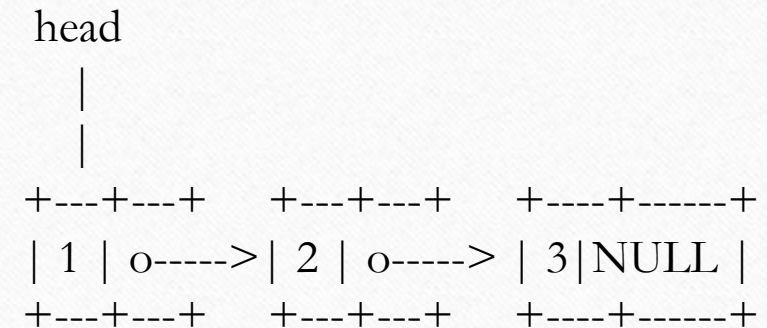


Singly Linked List (cont..)

```
head->data = 1;  
head->next = second;
```

```
second->data = 2;  
second->next = third;
```

```
third->data = 3;  
third->next = NULL;
```



SLL (Insert Node at Front)

```
Node* insert_at_first(Node* temp ,int data)
{
    Node* new_node = new Node();
    new_node->data = data;
    new_node->next = temp;
    return new_node;
}
```

Execution like;

```
head=insert_at_first(head , 12);
```

SLL Traversal

```
void traverse(Node* temp)
{
    while(temp->next != NULL)
    {
        cout<<temp->data<<endl;
        temp = temp->next;
    }
}
```

```

int main()
{
    Node* head = new Node;

    LL l1;

    int x;

    do{
        cout<<"Enter Choice: ";
        cin>>x;
        head = l1.create(head);
    } while (x==1);

    l1.traverse(head);
    system("pause");
    getchar();
    return 0;
}

```

```

class Node
{
public:
    int data;
    Node* next;
};

class LL
{
public:
    Node* create(Node* temp)
    {
        Node* new_node = new Node();
        int x;
        cin>>x;
        new_node->data = x;
        new_node->next = temp;
        return new_node;
    }

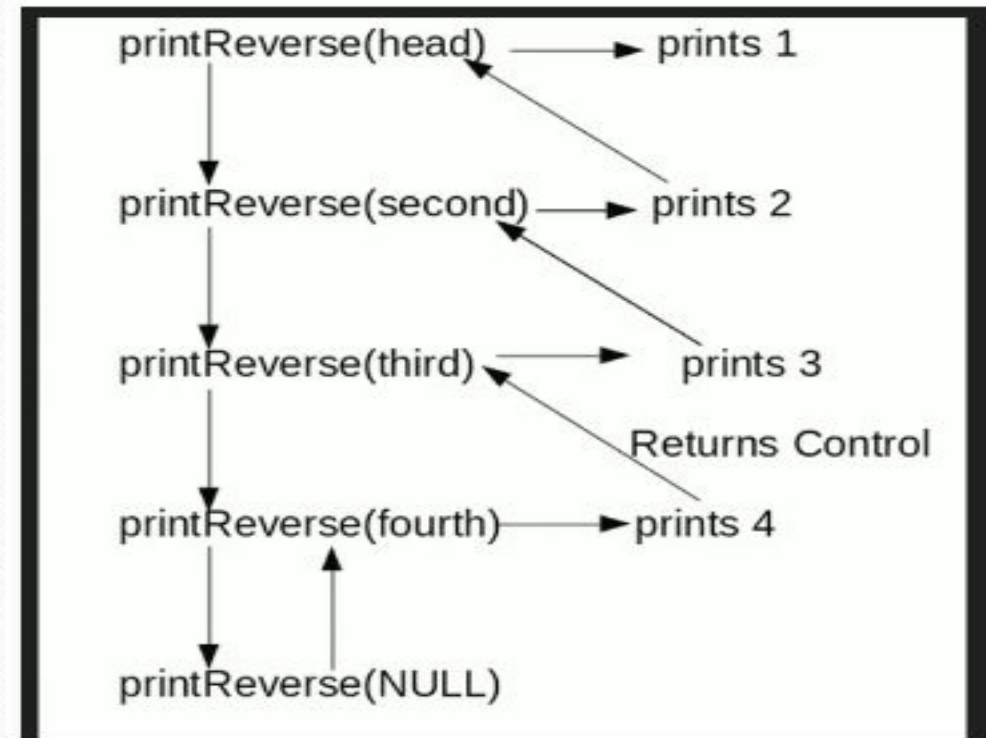
    void traverse(Node* temp)
    {
        while(temp != NULL)
        {
            cout<<temp->data<<endl;
            temp = temp->next;
        }
    }
};

```

LL Traversing Reverse Order (Recursive)

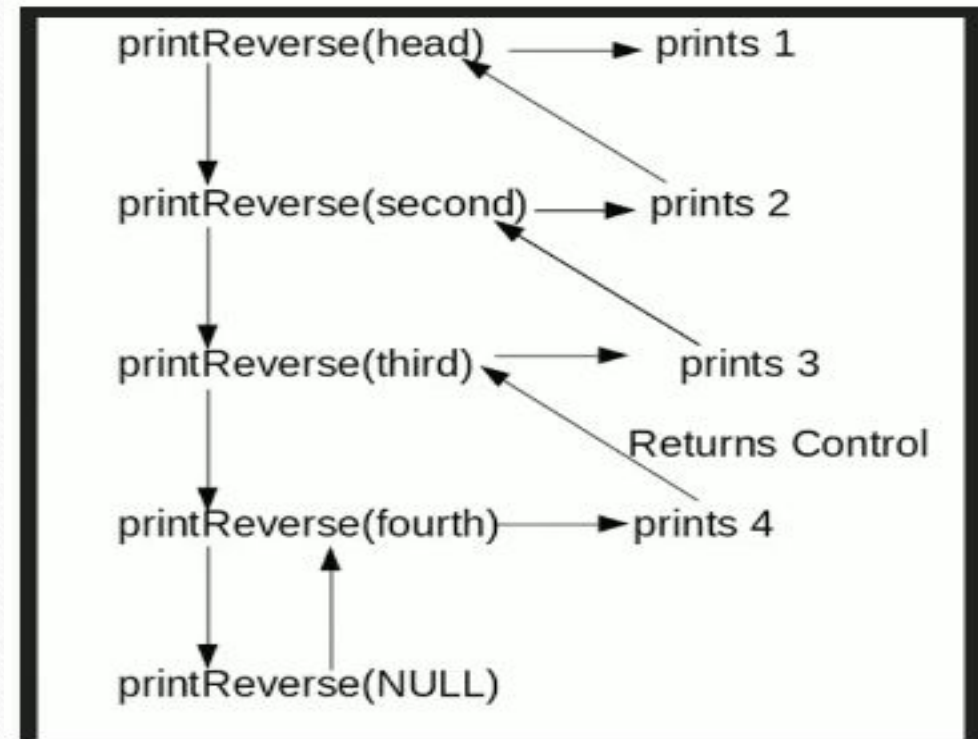
`printReverse(head)`

1. call print reverse for head->next
2. print head->data



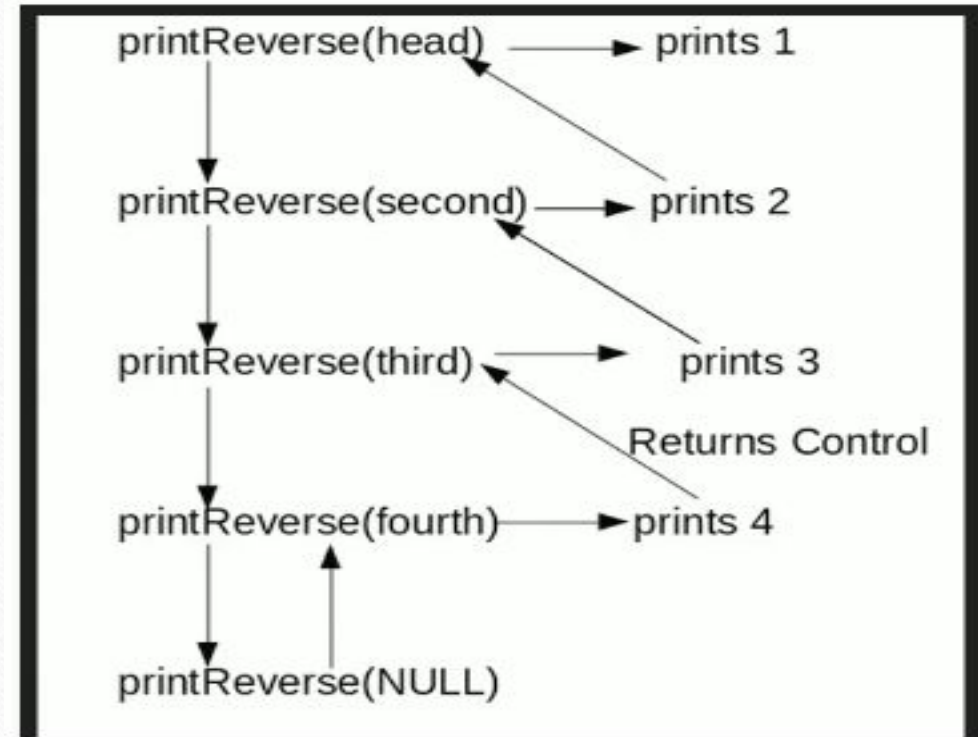
LL Traversing Reverse Order (Recursive)

```
void printReverse(Node* head)
{
    // Base case
    if (head == NULL)
        return;
    // print the list after head node
    printReverse(head->next);
    // After everything else is printed, print head
    cout << head->data << " ";
}
```



LL Traversing Reverse Order (Recursive)

- Base case in recursion: Without this, recursion would continue forever until program crash (segmentation fault). It's like telling the function: "Stop when you reach the end of the list."
- Prevents dereferencing null, If we did head->next when head is nullptr, the program would crash.
- This check makes sure we don't access memory that doesn't exist.

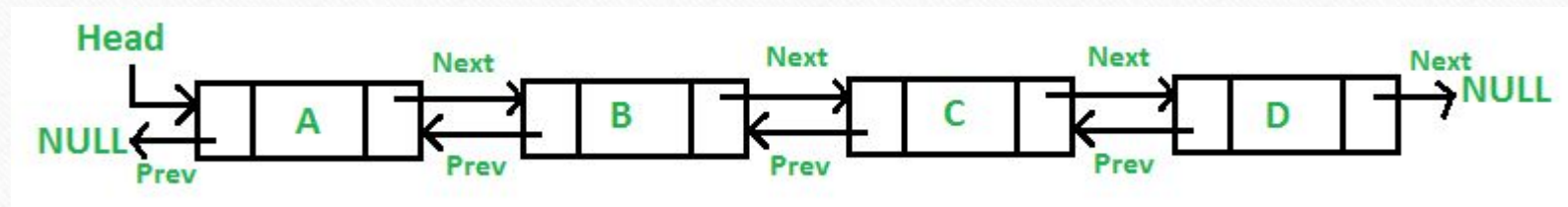


Complete Singly-Linked List Code

Shared!

Doubly Linked List

A Doubly Linked List (DLL) contains an extra pointer, typically called previous pointer, together with next pointer and data which are there in singly linked list.



Doubly Linked List

```
class Node
{
    int data;
    Node* next;
    Node* previous;
}
```

Advantages over singly linked list

- A DLL can be traversed in both forward and backward direction.
- The delete operation in DLL is more efficient if pointer to the node to be deleted is given.
- We can quickly insert a new node before a given node.

Disadvantages over singly linked list

- Every node of DLL Require extra space for an previous pointer.
- Additional operation to maintain previous pointer.

Task!

Construct a DDL consisting of 4 Nodes
just like we did for SLL

Task (Answer)

```
Node *head = new Node();
Node *second = new Node();
Node *third = new Node();
Node *fourth = new Node();

head->data = 1;
second->data = 2;
third->data = 3;
fourth->data = 4;

head->previous = NULL;
head->next = second;

second->previous = head;
second->next = third;

third->previous = second;
third->next = fourth;

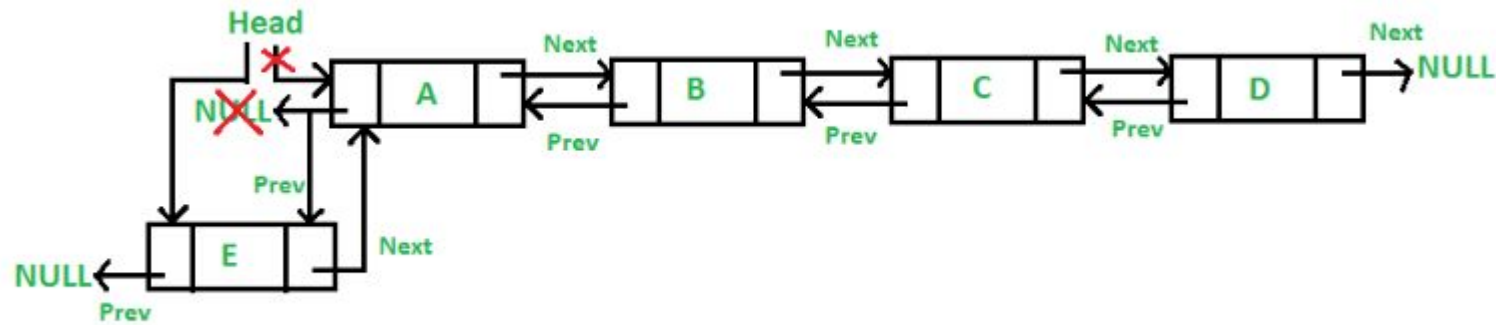
fourth->previous = third;
fourth->next = NULL;
```

DDL Traversing

```
void traverse1(Node* temp)
{
    while(temp != NULL)
    {
        cout<<temp->data<<endl;
        temp = temp->next;
    }
}
```

```
void traverse2(Node* temp)
{
    while(temp != NULL)
    {
        cout<<temp->data<<endl;
        temp = temp->previous;
    }
}
```

DLL (Add a node at the front)



DLL (Add a node at the front)

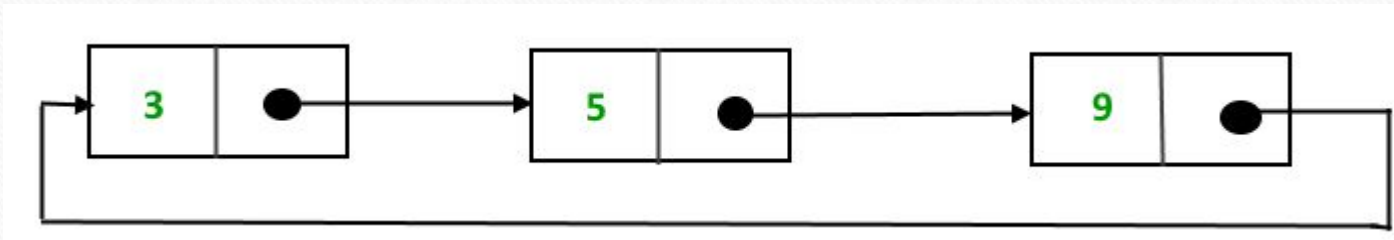
```
Node* insert_at_first(Node* temp ,int data)
{
    //Step 1
    Node* new_node = new Node();
    new_node->data = data;
    //Step2
    new_node->next = temp;
    new_node->previous = NULL;
    //Step3
    new_node->next = temp;
    temp->previous = new_node;

    return new_node;
}
```

Task!

- DLL (Add a node at the end)
- Add a node before a given node
- Add a node after a given node

Circular Single Linked List



Pros: Any node can be a starting point. We can traverse the whole list by starting from any point. We just need to stop when the first visited node is visited again.

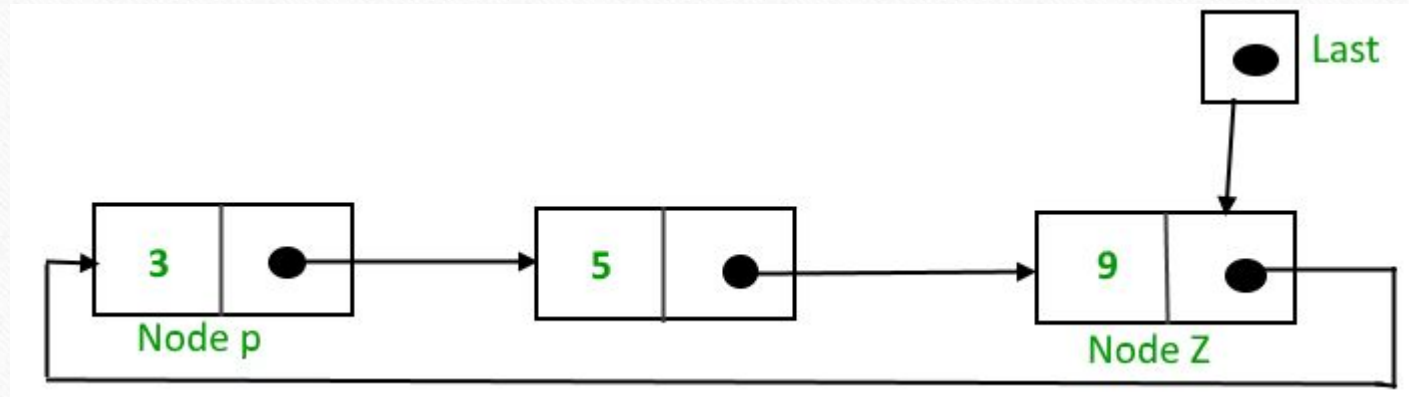
Circular Double Linked List



- Each node has **two pointers**: one to the **next** node and one to the **previous** node.
- The **last node** points back to the **first node**, making it circular.
- The **first node**'s prev pointer also points to the last node.

Implementation

- To implement a circular singly linked list, we take an external pointer that points to the last node of the list. If we have a pointer last pointing to the last node, then last -> next will point to the first node.

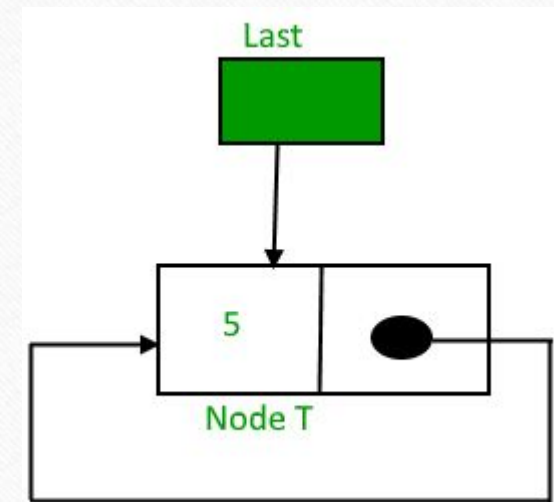


Insertion in an empty List

- Initially when the list is empty, *last* pointer will be NULL.



After inserting a node T,



Insert in Empty List

```
class Node
{
    public:
        int data;
        Node* next;
};
```

```
addToEmpty(int data)
{
    // Creating a node dynamically.
    Node *temp = new Node;

    // Assigning the data.
    temp -> data = data;
    last = temp;

    // Creating the link.
    last -> next = last;
}
```

Traversing

```
void traverse()
{
    Node *p;

    // If list is empty, return.
    if (last == NULL)
    {
        cout << "List is empty." << endl;
        return;
    }

    // Pointing to first Node of the list.
    p = last -> next;

    // Traversing the list.
    do
    {
        cout << p -> data << " ";
        p = p -> next;
    }
    while(p != last->next);
}
```

Insert at End

```
addEnd(int data)
{
    if (last == NULL)
        return addToEmpty(data);

    Node *temp = new Node;

    temp -> data = data;
    temp -> next = last -> next;
    last -> next = temp;
    last = temp;
}
```

Insert at Beginning

```
addBegin(int data)
{
    if (last == NULL)
        return addToEmpty(data);

    Node *temp = new Node;

    temp -> data = data;
    temp -> next = last -> next;
    last -> next = temp;
}
```

Linear Search In SLL

- Searching in Singly Linked List (SLL) Traversal:
- Only forward direction (using next pointer).
 - Process: Start at head.
 - Compare data of current node with the key.
 - If match found → return position.
 - Else move to next.
 - Stop when NULL is reached.
 - **Time Complexity: $O(n)$**

Linear Search In SLL

```
int searchSLL(Node* temp, int key) {  
    int pos = 1;           // start position  
    while (temp != nullptr) {  
        if (temp->data == key)  
            return pos;    // found → return current position  
        temp = temp->next;  // move to next node  
        pos++;             // move position forward  
    }  
    return -1;             // not found  
}
```

Execution:

```
int key = 30;  
int result = searchSLL(head, key);
```

Linear Search In DLL

- Searching in Doubly Linked List (DLL) Traversal:
- Both forward and backward (using next and prev).
- Process: Similar to SLL, but you can search:
- From head to tail OR From tail to head.
- Useful when you know the key might be near the end → faster to start from tail.
- **Time Complexity: $O(n)$**

Linear Search In DLL

```
int searchDLL(Node* temp, int key) {  
    int pos = 1;  
    while (temp != nullptr) {  
        if (temp->data == key)  
            return pos;  
        temp = temp->next;  
        pos++;  
    }  
    return -1;  
}
```

Execution:

```
int pos = searchDLL(head, 30);
```

Linear Search In CRLL

- **Searching in Circular Linked List (CRLL) Traversal:**
- Moves around in a circle → stop when you come back to head.
- Process: Start at head.
- Keep moving to next and compare with key.
- Stop when either: Key found → return position.
- You return back to head → key not present.
- **Time Complexity: $O(n)$**

Linear Search In CRL

-
- **Task: CODE Yourself**

Exercise!

- a. Given a SinglyLinkedList, write a function that remove all duplicate values from the list. Your function should handle the case that if two adjacent nodes are the same, the function should remove the second appearance of the duplicate value. [5]
- b. Given a Circular Singly Linked List(CSLL) with only head as a pointer to access it. Write a function that insert a given node at the middle of CSLL. You need to identify the hidden cases as well. [5]

```
void InsertAtMiddle (Node<T> *rhs) ;
```

Solution

```
void RemoveDuplicates(Node<T> * head)
{
    Node<T> * curr,*temp,*duplicate;
    curr=head;

    while(curr != 0 && curr->Next !=0)
    {
        temp=curr;

        while(temp->Next != 0)
        {
            if (curr->Data == temp->Next->Data)
            {
                duplicate=temp->Next;
                temp->Next=temp->Next->Next;
                duplicate->Next=0;
                delete(duplicate);
            }
            else
                temp=temp->Next;
        } // inner while
        curr= curr->Next;
    } // outer while
} // end of RemoveDuplicates
```

• • • •

```
void InsertAtMiddle(Node<T> *n)
{
    Node<T> * oneStep,*twoSteps;
    oneStep=twoSteps=head;

    while(twoSteps != 0)
    {
        oneStep= oneStep->Next;
        twoSteps= twoSteps->Next->Next;

        if(twoSteps->Next == head)
        {
            n->Next= oneStep->Next;
            oneStep->Next=n;
            break;
        }
        if(twoSteps->Next->Next == head)
        {
            n->Next= oneStep->Next;
            oneStep->Next=n;
            break;
        }
    } //infinite while
}
```